

Recursion

Recursion is nothing but the ability of a function to call itself, until the desired function is satisfied.

There must be an exclusive stopping condition statement within the body of a recursive function. Otherwise, the function enters an infinite loop.

Recursive definition and process

Consider the process of finding the factorial of a given number. Given a positive integer n , n factorial ($n!$) is defined as the product of all integers between ' n ' and ' 1 '.

We may therefore write the definition of this function as follows,

$$n! = 1, \text{ if } n = 0$$

$$n! = n * (n-1) * (n-2) * \dots * 1, \text{ if } n > 0$$

We may note that $n! = n * (n-1)!$

Using mathematical notation,

$$n! = 1, \text{ if } n = 0$$

$$n! = n * (n-1)!, \text{ if } n > 0$$

A definition which defines an object in terms of simpler case of itself is called a recursive definition.

When the recursive program is taken for execution, the recursive function calls will not be executed immediately.

They are pushed into the stack as long as the stopping condition is encountered, the recursive function calls are popped from the stack in the reverse order and executed one by one.

Ex:

```
int fact(n)
{
    int x, y;
    if (n==0) return (1);
    else
    {
        x=n-1;
        y=fact(x);
        return(n*y);
    }
}
```

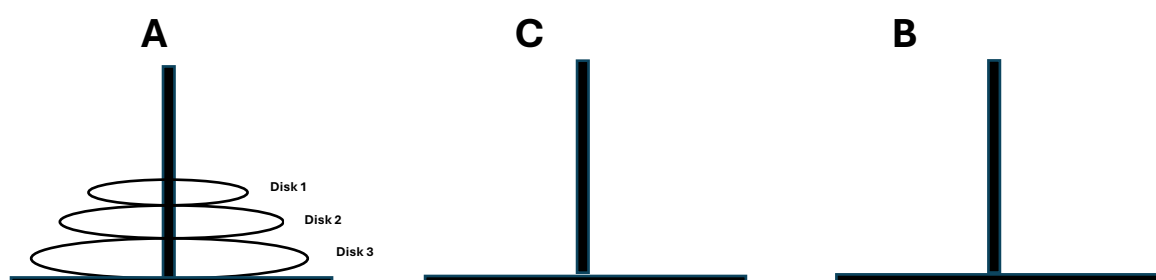
The Towers of Hanoi Problem:

The problem is stated as follows:

There will be three poles and n number of different sized disks. The disks are placed on the poles in such a way that larger disk is always below the smaller disk.

The objective is to move these n disks from source pole to destination pole by using auxiliary pole.

Only the top disk on any pole may be moved to any other pole and a larger disk should not be rest on the smaller disk and at a time only one disk can be moved.



Source Pole

Destination Pole

Auxiliary Pole

In this case the disks numbered 1,2,3 should be transferred from pole A (Source Pole) to pole C (Destination Pole) using pole B as Auxiliary Pole (temporary pole).

We may state a recursive solution to the Towers of Hanoi problem as follows:

To move n disks from Pole A to Pole C by using Pole B:

1. If $n=1$, move disk the single disk from A to C and Stop.
2. Move the top n-1 disks from A to B by using C as auxiliary.
3. Move the remaining nth disk from A to C.
4. Move the n-1 disks from B to C by using A as auxiliary.

Recursive 'C' code to solve the Towers of Hanoi Problem.

```
void Towers(int n, char src, char dst, char aux)
{
    if (n==1) printf ("Move disk 1 from %c to %c", src, dst);
    else
    {
        Towers(n-1, src, aux, dst);
        printf ("Move disk %d from %c to %c", n-1, src, dst);
        Towers(n-1, aux, dst, src);
    }
}
```

Example: $n=3$, output statements:

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Efficiency of Recursion:

In general, a non-recursive version of a program will execute, more efficiently in terms of time and space than a recursive version.

This is because in the case of iterative programs (non recursive programs) the local variables will not be pushed into the stack.

But in the case of recursive procedure, the compiler will push the local variables into the stack and once the stopping or terminating condition is encountered, the variables are popped from the stack.

We know that sometimes, a recursive solution is the most natural and logical way of solving a problem. For example, the iterative procedure to solve Towers of Hanoi problem is very difficult.

In such cases, it is better to write or transform, the recursive procedure into equivalent iterative procedure and vice-versa.

In the case of iterative program, the instructions will be executed in a fixed no. of times (for example n times).

In the case of recursive programs, we must specify explicitly, the stopping condition. Otherwise, the recursive call will enter infinite loop.

However, the recursion is useful in algorithms and data structures which are inherently recursive in nature.

Recursive program for multiplication of natural numbers

Consider the product $a*b$, where a and b are +ve integers, may be defined as 'a' added itself 'b' times.

$$a * b = 1 \text{ if } b = 1$$

$$a * b = a * (b-1) + a \text{ if } b > 1$$

To evaluate $6 * 3$ by the definition, we first evaluate, $6 * 2$ and then add 6 to it.

To evaluate $6 * 2$, we first evaluate $6 * 1$, and then add 6

Multiplication of two numbers:

```
int mult(int a, int b)
{
    int c,d;
    if(b==1)
        return(a);
    c=b-1;
    d=mult(a,c);
    return(d+a);
}
```

```
}
```

GCD of two numbers

```
int gcd(int a, int b)
{
    int rem;
    if(b==0)
        return(a);
    rem=a%b;
    gcd(b,rem);
}
```